

**MAŁOPOLSKA UCZELNIA PAŃSTWOWA
IMIENIA ROTMISTRZA WITOLDA PILECKIEGO
W OŚWIĘCIMIU**

Instytut: Nauk Inżynieryjno-Technicznych

Kierunek studiów: Informatyka

Poziom studiów: Studia pierwszego stopnia

Moduł specjalnościowy: Informatyka w przedsiębiorstwie

Forma studiów: stacjonarna

Rok studiów/semestr: rok III, semestr VI

Krystian Zając

nr albumu 8795

**PROJEKT APLIKACJI MOBILNEJ ZINTEGROWANEJ
Z BAZĄ DANYCH I URZĄDZENIAMI IoT**

Projekt na przedmiot systemy mobilne

Prowadzący: mgr inż. Artur Pelo

Oświęcim 2026

Abstrakt

Imię i nazwisko autora projektu	Krystian Zając
Rok urodzenia autora projektu	2003
Stopień/tytuł zawodowy, imię i nazwisko prowadzącego przedmiotu	mgr inż. Artur Pelo
Instytut/Kierunek/Poziom/ Moduł specjalnościowy	Instytut Nauk Inżynieryjno-Technicznych / Informatyka / Inżynierskie / Informatyka w przedsiębiorstwie
Data realizacji projektu	od 29.04.2026 do 13.05.2026
Nazwa przedmiotu	systemy mobilne
Rok studiów/semestr	rok III, semestr VI

Tytuł projektu	Projekt aplikacji mobilnej zintegrowanej z bazą danych
Słowa kluczowe (max 5)	informatyka, aplikacje mobilne, bazy danych
Streszczenie (max 1400 znaków)	Tekst streszczenia

Spis treści

Wstęp	4
1. Cel i zakres pracy	5
1.1. Cel pracy	5
1.2. Cele szczegółowe	5
1.3. Zakres pracy	5
2. Analiza wymagań i projekt rozwiązania	6
2.1. Wymagania funkcjonalne	6
2.2. Wymagania нефункционалне	6
2.3. Architektura rozwiązania	6
2.4. Warstwa sprzętowa	6
2.5. Warstwa serwera	7
2.6. Warstwa bazy danych	7
2.7. Warstwa aplikacji mobilnej	7
3. Opis działania aplikacji	8
3.1. Środowisko i narzędzia	8
3.2. Struktura projektu	8
3.3. Interfejs użytkownika	8
3.4. Pobieranie danych z REST API	10
3.5. Automatyczne odświeżanie co 10 sekund	10
3.6. Logika wyświetlania aktualnej temperatury	10
3.7. Wyświetlanie dziennika	10
4. Implementacja i uruchomienie	11
4.1. Wymagania sprzętowe	11
4.2. Instrukcja instalacji	11
4.3. Instrukcja użytkowania	11
5. Testy i weryfikacja	12
5.1. Testy funkcjonalne	12
5.2. Testy нефункционалне	12
5.3. Wyniki testów	12
Podsumowanie	13
Spis rysunków	14
Spis tabel	15
Spis listingów	17

Wstęp

Rozwój technologii Internetu Rzeczy (IoT) umożliwia zdalne monitorowanie parametrów środowiskowych w czasie rzeczywistym. Jednym z najczęściej mierzonych parametrów jest temperatura, wykorzystywana zarówno w systemach domowych, jak i przemysłowych. Niniejszy projekt przedstawia implementację systemu składającego się z mikrokontrolera ESP32 WROOM z czujnikiem temperatury DHT11, lokalnej bazy danych MySQL (XAMPP) oraz aplikacji mobilnej na system Android.

System umożliwia zapisywanie pomiarów temperatury do bazy danych oraz ich odczyt i prezentację w aplikacji mobilnej w postaci aktualnej wartości oraz dziennika historycznego.

1. Cel i zakres pracy

1.1. Cel pracy

Celem pracy jest zaprojektowanie i implementacja aplikacji mobilnej w środowisku Android Studio, zintegrowanej z bazą danych MySQL, umożliwiającej odczyt i prezentację danych temperatury pochodzących z czujnika DHT11 podłączonego do modułu ESP32 WROOM.

1.2. Cele szczegółowe

W ramach realizacji projektu wyznaczono następujące cele szczegółowe:

- konfiguracja modułu ESP32 WROOM do odczytu danych z czujnika DHT11,
- implementacja mechanizmu zapisu danych (data, godzina, temperatura) do bazy MySQL,
- utworzenie lokalnego serwera bazy danych przy użyciu XAMPP,
- opracowanie interfejsu API do komunikacji aplikacji z bazą danych,
- zaprojektowanie aplikacji mobilnej w Android Studio,
- implementacja mechanizmu pobierania danych z bazy (REST API, JSON),
- wyświetlanie aktualnej temperatury oraz dziennika pomiarów w aplikacji,
- przeprowadzenie testów funkcjonalnych i нефункциональных.

1.3. Zakres pracy

Zakres pracy obejmuje:

- konfigurację sprzętową (ESP32 + DHT11),
- konfigurację środowiska XAMPP i MySQL,
- implementację RestAPI,
- stworzenie aplikacji mobilnej Android,
- testowanie systemu.

2. Analiza wymagań i projekt rozwiązania

Rozdział przedstawia wymagania oraz projekt rozwiązania. Opis obejmuje zarówno aspekty funkcjonalne, jak i нефункционалне, a także opis architektury.

2.1. Wymagania funkcjonalne

Do kluczowych wymagań funkcjonalnych należą:

- aplikacja musi łączyć się z serwerem opartym o SpingBoot,
- zapis danych do lokalnej relacyjnej bazy SQL lub nierelacyjnej na serwerze,
- aplikacja musi pobierać aktualny pomiar temperatury,
- aplikacja musi wyświetlać dziennik zapisów,
- dane muszą być prezentowane w sposób czytelny,
- integracja z urządzeniami RestAPI,
- przygotowanie dokumentacji zgodnej z wymaganiami edytorskimi.

2.2. Wymagania нефункционалне

Rozwiązanie powinno spełniać następujące wymagania jakościowe:

- kompatybilność z systemami Android 8.0+,
- czas odpowiedzi API mniejszy niż 300ms,
- dostępność systemu większa niż 99%,
- szyfrowanie SSL/TLS dla transmisji danych,
- dostępność zgodna z WCAG 2.1 na poziomie AA.

2.3. Architektura rozwiązania

Architektura systemu została zaprojektowana w modelu warstwowym, w którym poszczególne komponenty odpowiadają za ściśle określone zadania. Rozdzielenie funkcjonalności na warstwy zwiększa czytelność projektu, ułatwia jego rozwój oraz umożliwia niezależne modyfikowanie poszczególnych elementów systemu. Całość rozwiązania opiera się na komunikacji sieciowej w modelu klient–serwer.

System składa się z czterech głównych warstw: warstwy sprzętowej, warstwy komunikacyjnej i serwerowej, warstwy bazy danych oraz warstwy aplikacji mobilnej.

2.4. Warstwa sprzętowa

- Moduł ESP32 WROOM,
- Czujnik DHT11,

ESP32 odczytuje temperaturę i wysyła dane metodą HTTP POST do serwera.

2.5. Warstwa serwera

- XAMPP (Apache + MySQL),
- SpringBoot + RestAPI,

API odpowiada za:

- otrzymywanie danych z ESP32,
- zapisywanie do MySQL,
- udostępnianie danych aplikacji mobilnej w formacie JSON.

2.6. Warstwa bazy danych

Tabela 1

Struktura tabeli temperatura

Pole	Typ danych
id	int
data	date
godzina	int
temperatura	int

Źródło: opracowanie własne

2.7. Warstwa aplikacji mobilnej

Aplikacja Android:

- komunikacja z API,
- przetwarzanie JSON,
- wyświetlanie danych w UI.

3. Opis działania aplikacji

Rozdział opisuje logikę funkcjonowania aplikacji poprzez opisanie jej poszczególnych funkcjonalności oraz przedstawienie kodu.

3.1. Środowisko i narzędzia

- Android Studio,
- Język Java,
- ESP32 - Arduino IDE,
- XAMPP (Apache, MySQL),
- Docker

3.2. Struktura projektu

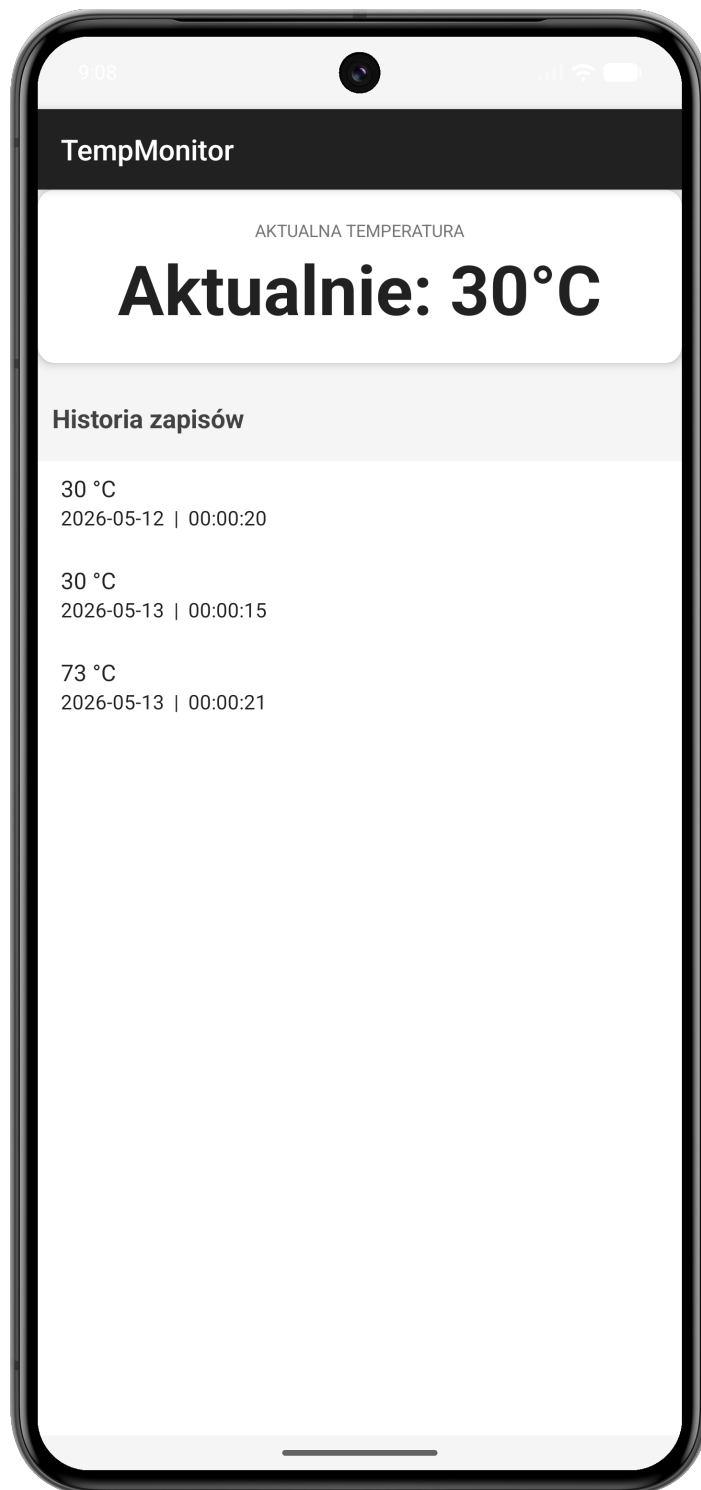
Struktura projektu składa się z plików projektu aplikacji Android Studio, projektu aplikacji serwera SpringBoot w IntelliJ, oraz bazy danych implementowanej za pomocą Docker Desktop.

3.3. Interfejs użytkownika

Sekcja "Aktualna Temperatura": Umieszczona na samej górze wewnątrz karty. Wyświetla największą czcionką ostatni odczyt z bazy danych, co pozwala na natychmiastowe sprawdzenie stanu termometru.

Sekcja "Historia zapisów": Poniżej karty znajduje się dynamiczna lista. Każdy wiersz zawiera temperaturę oraz dokładną datę i godzinę wykonania pomiaru.

Jak pokazano na rysunku 1, poniżej interfejs został zaprojektowany z myślą o czytelności i nowoczesnym wyglądzie.



Rysunek 1. Interface aplikacji

Źródło: opracowanie własne przy użyciu emulatora Android Studio

3.4. Pobieranie danych z REST API

Aplikacja korzysta z biblioteki Retrofit do asynchronicznego pobierania danych. Oznacza to, że pobieranie odbywa się w tle i nie zawiesza ekranu użytkownika.

Listing 1. Pobieranie danych z API (Fragment ApiService.java)

```
1 @GET("api/pomiary")
2 Call<List<Dane>> getWszystkiePomiary();
```

3.5. Automatyczne odświeżanie co 10 sekund

Zastosowano mechanizm Handler i Runnable, który tworzy pętlę czasową. Funkcja ta jest inteligentna: uruchamia się, gdy aplikacja jest widoczna, i zatrzymuje, gdy użytkownika zminimalizuje.

Listing 2. Odświeżanie (Fragment MainActivity.java)

```
1 refreshRunnable = new Runnable() {
2     @Override
3     public void run() {
4         pobierzDaneZSerwera(); // Wywołanie API
5         handler.postDelayed(this, 10000); // Ponowne uruchomienie za 10s
6     }
7 };
```

3.6. Logika wyświetlania aktualnej temperatury

Aplikacja nie potrzebuje osobnego zapytania o jedną temperaturę. Pobiera całą listę, a następnie wybiera z niej ostatni element, aby wyświetlić go jako stan aktualny.

Listing 3. Wyświetlanie "Aktualnej temperatury"(Fragment MainActivity.java)

```
1 if (!daneList.isEmpty()) {
2     Dane ostatni = daneList.get(daneList.size() - 1); // Pobranie najnowszego
    wpisu
3     tvCurrentTemp.setText(ostatni.getTemperatura() + "'C");
4     Collections.reverse(daneList); // Odwrocenie listy dla historii (najnowsze
    na gorze)
5 }
```

3.7. Wyświetlanie dziennika

Adapter bierze surowe dane i wstrzykuje je w odpowiednie pola tekstowe w layoucie.

Listing 4. Wyświetlanie dziennika (Fragment DaneAdapter.java)

```
1 @Override
2 public void onBindViewHolder(@NonNull ViewHolder holder, int position) {
3     Dane d = lista.get(position);
4     holder.text1.setText(d.getTemperatura() + "'C");
5     holder.text2.setText(d.getData() + "|||" + d.getGodzina());
6 }
```

4. Implementacja i uruchomienie

Instrukcja zawiera informacje potrzebne do instalacji i użytkowania aplikacji.

4.1. Wymagania sprzętowe

Po stronie użytkownika:

- Smartfon z Android 8.0+
- Połączenie Wi-Fi

Po stronie serwera:

- Komputer z XAMPP
- ESP32 WROOM
- Czujnik DHT11
- Zasilanie 5V

4.2. Instrukcja instalacji

- Połącz komputer (serwer) i telefon do tej samej sieci Wi-Fi.
- W pliku MainActivity.java w Android Studio wpisz to IP w linii `.baseUrl("http://:8080/")` jeśli używasz emulatora nic nie zmieniaj.
- Uruchom bazę danych Docker/XAMPP oraz aplikację SpringBoot.
- Podłącz telefon do komputera i w Android Studio kliknij zieloną strzałkę Run.

4.3. Instrukcja użytkowania

Po uruchomieniu aplikacji użytkownik otrzymuje natychmiastowy dostęp do aktualnej temperatury oraz chronologicznej historii wszystkich zapisów. Dane w programie aktualizują się automatycznie co dziesięć sekund, co zapewnia stały podgląd na najświeższe odczyty bez konieczności ręcznego odświeżania ekranu. System inteligentnie zarządza baterią, wstrzymując pobieranie informacji w momencie zminimalizowania aplikacji i wznowiając pracę zaraz po jej ponownym otwarciu.

5. Testy i weryfikacja

Rozdział przedstawia sposób sprawdzenia poprawności działania aplikacji oraz ocenę uzyskanych rezultatów. Testy skupiają się na weryfikacji poprawności działania oraz zgodności z wymaganiami funkcjonalnymi i нефункциональными.

5.1. Testy funkcjonalne

Tabela 2

Tabela wyników testów funkcjonalnych

Test	Oczekiwany Wyniki	Wynik
Połączenie z API	Dane zwracane w JSON	Pozytywny
Wyświetlanie aktualnej temp.	Widoczna poprawna wartość	Pozytywny
Wyświetlanie dziennika	Lista rekordów	Pozytywny

Źródło: opracowanie własne

5.2. Testy нефункционаalne

Test wydajności

- Czas odpowiedzi API: akceptowalny
- Aplikacja reaguje płynnie.

Test kompatybilności

- Android 8, 10, 13 – działanie poprawne.

5.3. Wyniki testów

System działa poprawnie i spełnia wszystkie założenia funkcjonalne. Komunikacja między ESP32, bazą danych i aplikacją mobilną przebiega bez błędów. Dane są poprawnie zapisywane i odczytywane. Interfejs użytkownika jest czytelny i intuicyjny.

Podsumowanie

Zrealizowany projekt umożliwia zdalne monitorowanie temperatury z czujnika DHT11 za pomocą aplikacji mobilnej Android. System integruje technologie IoT, bazę danych MySQL oraz aplikację mobilną, tworząc spójne rozwiązanie do monitorowania parametrów środowiskowych.

Projekt może zostać w przyszłości rozszerzony o:

- wykresy temperatur,
- powiadomienia o przekroczeniu progów,
- obsługę wielu czujników,
- wdrożenie na serwerze zewnętrznym.

Spis rysunków

<i>Rysunek 1.</i> Interface aplikacji	9
---	---

Spis tabel

Tabela 1. Struktura tabeli temperatura	7
Tabela 2. Tabela wyników testów funkcjonalnych	12

Spis listingów

Listing 1. Pobieranie danych z API (Fragment ApiService.java)	10
Listing 2. Odświeżanie (Fragment MainActivity.java)	10
Listing 3. Wyświetlanie "Aktualnej temperatury"(Fragment MainActivity.java) . . .	10
Listing 4. Wyświetlanie diennika (Fragment DaneAdapter.java)	10